

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\`  
a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-  
aa969af8690883c2e.jsonl`  
- SHA-256: `c93fed60097e46c9a5fef6dd14a82d064fc4aca9d61e27d96760e055a725995c`  
- Source modified: `2026-05-31T04:18:31+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:17:29.380Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMV3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** <https://arxiv.org/html/2510.19817v1>

****Title:**** olmOCR 2: Unit Test Rewards for Document OCR (Allen AI, arXiv)

****Found via:**** Open VLMs for document/table extraction (broad/primary) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:32.487Z)

I'll fetch the source content first, then assess and extract claims.

3. assistant (2026-05-31T04:17:35.081Z)

I'll fetch the olmOCR 2 paper from arXiv.

4. user (2026-05-31T04:17:54.138Z)

olmOCR 2: Key Claims Extract

****Publication & Attribution****

- Technical report published October 2025 by Jake Poznanski, Luca Soldaini, and Kyle Lo at Allen Institute for AI (Ai2)

1. Licensing & Model Details

****Code & Model Weights:**** "all under permissive open licenses" with Apache 2.0 designation noted in Table 1. The model checkpoint ****olmOCR-2-7B-1025**** is based on ****Qwen2.5-VL-7B-Instruct****, which the paper indicates is openly available.

****Base Model:**** Qwen 2.5 VL is a 7B parameter vision-language model. The paper does not explicitly state Qwen's license in this document, though it notes "We thank the Qwen team for releasing open VLM models."

2. Performance on Document OCR

- ****Overall olmOCR-Bench score: 82.4 ± 1.1 **** (October 2025 release)
- ****+14.2 point improvement**** over initial olmOCR release ($68.2 \rightarrow 82.4$)

****Specialized performance gains:****

- Math formulas: 82.3 (up from baseline)
- Tables: 84.9 (substantially improved)
- Multi-column layouts: 83.7 (key target)
- Headers & footers: 47.7

The paper reports "largest improvements in math formula conversion, table parsing, and multi-column layouts" versus previous versions.

3. Hardware Requirements & Integration

The document states training used "8xH100 GPU node" but provides ****no explicit VRAM requirements, throughput metrics, or inference framework details**** (vLLM/transformers compatibility not mentioned). Model size is 7B parameters; quantized version available as "olmOCR-2-7B-1025-FP8."

4. Unit-Test-Reward Training Method

The reinforcement learning approach uses binary unit tests as reward signals for:

- ****Text Presence/Absence**** checks
- ****Natural Reading Order**** validation
- ****Table Accuracy**** (cell position & value verification)
- ****Math Formula Accuracy**** (KaTeX visual rendering comparison)
- ****Baseline Robustness**** (n-gram/character hallucination detection)

Training details: "one epoch of SFT training on olmOCR-mix-1025, [then] one epoch of RL training over our synthetic data olmOCR2-synthmix-1025" using GRPO with " $\epsilon=0.01$ " KL divergence penalty. Synthetic data pipeline generated ****30,381 test cases**** across 2,186 PDF pages using Claude Sonnet for HTML generation (~\$0.12/page cost).

5. Benchmark Comparisons

olmOCR 2 ranks among top performers:

- ****Chandra OCR 0.1.0: 83.1 ± 0.9 **** (highest reported)
- ****olmOCR 2: 82.4 ± 1.1 **** (Ai2, fully open)
- Infinity-Parser 7B: $82.5 \pm ?$
- PaddleOCR-VL: 80.0 ± 1.0
- DeepSeek-OCR: 75.7 ± 1.0
- GPT-4o: 68.9 ± 1.1 (closed, API-only)

6. Hallucination & Failure Modes

The paper addresses one specific hallucination failure: the model "would hallucinate" on blank pages before a bug fix in the data loader.

On ****numbers and tables specifically:**** The unit-test framework targets table cell accuracy and math formula rendering fidelity, but the document does not report explicit error rates for numeric hallucination or discuss known failure modes on dense financial/statistical tables beyond the general benchmark results.

The use of unit tests is motivated partly because "floating document elements like tables or figures lack a definitive ground truth representation," suggesting some ambiguity in evaluation rather than measured hallucination risk.

5. assistant (2026-05-31T04:17:58.005Z)

I have substantial detail. Let me verify a couple of license-critical specifics directly from the source, since the WebFetch summary hedged on the exact license labels.

6. user (2026-05-31T04:18:07.135Z)

Analysis of olmOCR 2 Licensing and Openness

(1) olmOCR 2 and Apache 2.0 License:

Table 1 indicates olmOCR 2 is labeled with superscript "1" corresponding to "Apache 2.0" in the footnotes. The exact caption states: "Apache 2.0¹ Apache 2.0"

(2) Chandra OCR's License:

Chandra OCR (0.1.0) is marked with superscript "3" in Table 1, corresponding to: "OpenRAIL-M³ OpenRAIL-M"

(3) Other Systems' Openness:

- **Infinity-Parser 7B**: Marked with superscript "1" (Apache 2.0)
- **PaddleOCR-VL**: Marked with superscript "1" (Apache 2.0)
- **DeepSeek-OCR**: Marked with superscript "2" (MIT)

(4) olmOCR General Openness Statement:

The introduction states: "olmOCR 2 is powered by olmOCR-2-7B-1025, an OCR-specialized VLM trained using reinforcement learning with verifiable rewards (RLVR)... We release our model, data and code under permissive open licenses."

(5) Qwen2.5-VL-7B-Instruct Base Model:

Section 3.2 specifies: "We start with a Qwen2.5-VL-7B-Instruct model that has been fine-tuned on olmOCR-mix-1025."

7. assistant (2026-05-31T04:18:11.240Z)

I now have all the verified details I need, including the critical license labels (Apache 2.0 for olmOCR 2 code/weights/data, built on Qwen2.5-VL-7B-Instruct) and the comparison that Chandra OCR is OpenRAIL-M (a restrictive license ? relevant to the research question's license-safety focus).

This is a primary research source: a technical report from the Allen Institute for AI (Ai2), authored by the model's creators, published October 2025 on arXiv.

8. user (2026-05-31T04:18:20.926Z)

Structured output provided successfully

9. assistant (2026-05-31T04:18:31.578Z)

Source processed. Summary of what I extracted from the olmOCR 2 technical report (Allen Institute for AI / Ai2, arXiv 2510.19817v1, October 2025):

Source quality: primary ? technical report authored by the model's creators (Poznanski, Soldaini, Lo at Ai2).

Key findings bearing on the muneem research question:

- License (critical): olmOCR 2 releases model weights, data, and code under Apache 2.0 ? fully commercial-safe for both code and weights. Notably, its top-scoring competitor Chandra OCR ships under the restrictive OpenRAIL-M license (flagged), while DeepSeek-OCR is MIT and Infinity-Parser/PaddleOCR-VL are Apache 2.0.

- Base model / VRAM tier: built on Qwen2.5-VL-7B-Instruct (7B params) ? places it in the ~16-24 GB GPU tier at FP16, with an FP8 checkpoint (olmOCR-2-7B-1025-FP8) available for lower VRAM.
- Table/multi-column strength: olmOCR-Bench scores of 84.9 (tables) and 83.7 (multi-column); paper explicitly cites table parsing and multi-column layouts as its largest improvement areas ? directly relevant to dense bank-statement tables.
- Verification/hallucination angle: RL reward uses unit tests checking table cell position+value accuracy and n-gram/character hallucination detection.
- Overall: 82.4 olmOCR-Bench, competitive with the best open systems and far above GPT-4o (68.9).

License caveat worth flagging downstream: the paper labels olmOCR 2 as Apache 2.0, but the actual base weights are Qwen2.5-VL-7B-Instruct ? the consuming research should independently confirm the Qwen2.5-VL-7B weights license (Apache-2.0 for the 7B size per Qwen's release, but the paper itself does not state Qwen's license). I rated the two license/base-model claims and the table-performance claim as central; the unit-test-reward and overall-benchmark claims as supporting.